

Lab 3

Ishanvi Verma

2025-06-29

Short Answer Question

Please response in 3-4 sentences.

Question 1

Why do you think the concept of a p-value is so often misunderstood?

P-value is the probability of obtaining results as unusual or extreme as the observed results given that the null-hypothesis

Question 2

Describe the advantages and disadvantages of single-blind vs. double-blind studies. What role does convenience play in experimental design?

Single blind is when the subject is unaware of the treatment they are getting during an experiment. Double blind is when both the subject and the facilitator are unaware of the treatment they are receiving from the experiment. In terms of convenience single blind studies are better because they are more affordable since double blind studies requires someone outside of the group of scientists to facilitate the study.

Question 3

Give an example of a one-way test. Give another example of a two-way test.

One-way test example could be how does grade level affect sat test scores. Two way test is like how does a student's grade level and time spent studying affect test scores.

Question 4

We are testing if women get promoted less than men in a business setting. Write out the null and alternative hypothesis below.

H_0 : Women are promoted as much as men in a business setting.

H_A : Women get promoted less than men in a business setting.

Question 5

What does this comic tell you about multiple testing?

This shows how at times multiple t-tests are not as good as an ANOVA test because it inflates the alpha value giving you results that are not actually statistically significant.

Coding Question

Section 1: Set Up

Load tidyverse.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Set your seed to a value of your choosing.

```
set.seed(100)
```

Section 2: Functions

Task 1: Write a function that takes parameters x and y and returns their product. Test it on values of your choosing.

```
product_function <- function(x,y) {
  product = x*y
}

print(product_function(2,5))
```

```
## [1] 10
```

Task 2: Write a function that takes two vectors and returns their two-sample t-statistic using the following formula: $t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$. Test it on values of your choosing.

```

t_stats = function(x,y) {
  avg1 = mean(x)
  avg2 = mean(y)
  sd1 = sd(x)
  sd2 = sd(y)
  length1 = length(x)
  length2 = length(y)
  t_stats = (avg1-avg2)/(sqrt((sd1)^2/(length1)) + ((sd2)^2/(length2)))
}

x=c(1,3,5)
y=c(2,4,6)
print(t_stats(x,y))

```

```
## [1] -0.4019238
```

Task 3: Write a function that takes a vector and returns a single bootstrap sample. You will use this function in section 5.

```

bootstrap_sample <- function(x) {
  mean(x, size = length(x), replace = TRUE) }

```

Section 3: For Loops and If Statements

Task 1: Create the for-loop version of `seq(0, 50, by = 5)`

```

for (i in 0:50){
  if (i %%5 == 0){
    print(i)
  }
}

```

```

## [1] 0
## [1] 5
## [1] 10
## [1] 15
## [1] 20
## [1] 25
## [1] 30
## [1] 35
## [1] 40
## [1] 45
## [1] 50

```

Task 2: Print out “hello” seven times using a for-loop.

```

for(i in 0:7) {
  print("hello")
}

```

```
## [1] "hello"
## [1] "hello"
## [1] "hello"
## [1] "hello"
## [1] "hello"
## [1] "hello"
## [1] "hello"
## [1] "hello"
```

Task 3: Create a for-loop that only prints out prime numbers from 1 to 100.

Hint: you might want to write a function called `is_prime()` that checks if a number is prime.

```
is_prime <- function(x) {
  if (x <= 1) {
    return(FALSE)
  }
  if (x == 2) {
    return(TRUE)
  }
  for (i in 2:(x-1)) {
    if (x %% i == 0) {
      return(FALSE)
    }
  }
  return(TRUE)
}

# Print all primes from 0 to 100
for (i in 0:100) {
  if (is_prime(i)) {
    print(i)
  }
}
```

```
## [1] 2
## [1] 3
## [1] 5
## [1] 7
## [1] 11
## [1] 13
## [1] 17
## [1] 19
## [1] 23
## [1] 29
## [1] 31
## [1] 37
## [1] 41
## [1] 43
## [1] 47
## [1] 53
## [1] 59
## [1] 61
## [1] 67
```

```
## [1] 71
## [1] 73
## [1] 79
## [1] 83
## [1] 89
## [1] 97
```

Task 4: Write a function that takes age as a parameter and returns “Of voting age” if it’s at least 18 and “Not of voting age” otherwise.

```
age <- function(x) {
  if(x >= 18){
    return("Of voting age")
  }
  return("Not of voting age")
}
```

Task 5: Write a function that converts the letters R,O,Y,G,B,I,V into their respective colors using case_when.

```
roygbiv_converter <- function(x) {
  x <- as.character(x)
  case_when(
    x == "R" ~ "red",
    x == "O" ~ "orange",
    x == "Y" ~ "yellow",
    x == "G" ~ "green",
    x == "B" ~ "blue",
    x == "I" ~ "darkblue",
    x == "V" ~ "violet",
  )
}
#print(roygbiv_converter("R"))
```

Task 6: Write a function that converts the integers 1 through 12 into months using case_when.

```
number_converter <- function(x) {
  case_when (
    x == 1 ~ "Jan",
    x == 2 ~ "Feb",
    x == 3 ~ "Mar",
    x == 4 ~ "Apr",
    x == 5 ~ "May",
    x == 6 ~ "Jun",
    x == 7 ~ "Jul",
    x == 8 ~ "Aug",
    x == 9 ~ "Sept",
    x == 10 ~ "Oct",
    x == 11 ~ "Nov",
    x == 12 ~ "Dec",
  )
}
#print(number_converter(1))
```

Section 4: The Birthday Problem

If somebody asked you in a room full of 25 people, “What is the probability that two people have matching birthdays?”, what would you say? The answer is higher than you might expect! Let’s use for-loops and functions to figure it out.

First, generate 25 samples from 365 days of the year. (Simply use integers to represent these dates.)

```
bday = sample(c(1:365), size=25, replace = TRUE)
#bday
```

Did you sample with or without replacement above? Why?

We sample with replacement because it is possible for two people to have the same birthday.

What assumptions are we making with randomly sampling 365 integers to represent the birthdays of 25 people?

We can assume that there is no leap year so the exact size of the year is 365 days.

Next, check if any birthdays are duplicated using `sum()` and `duplicated()`.

```
sum(duplicated(bday)) > 0
```

```
## [1] FALSE
```

Now, write a function to perform an experiment where we sample a group of random birthdays, and determine if anybody repeats.

```
duplicate_bday <- function() {
  return (sum(duplicated(bday)) > 0)
}
```

Let’s run this at scale! We’ll `replicate()` the experiment 5000 times:

```
results <- replicate(5000, duplicate_bday())
```

Calculate the probability of a matching birthday

```
group_size <- 25

duplicate_bday <- function() {
  bday <- sample(1:365, group_size, replace = TRUE)
  return(sum(duplicated(bday)) > 0)
}

results <- replicate(5000, duplicate_bday())
prob <- mean(results)
print(prob)
```

```
## [1] 0.5608
```

Section 5: Bootstrap

Task 1: Create a bootstrap simulation using 30 datapoints from a normal distribution.

Create a set of 30 datapoints from a normal distribution with a mean and sd of your choosing.

Hint: use `rnorm()`.

```
fake_data1 <- rnorm(30, mean = 5, sd = 3)
print(x)
```

```
## [1] 1 3 5
```

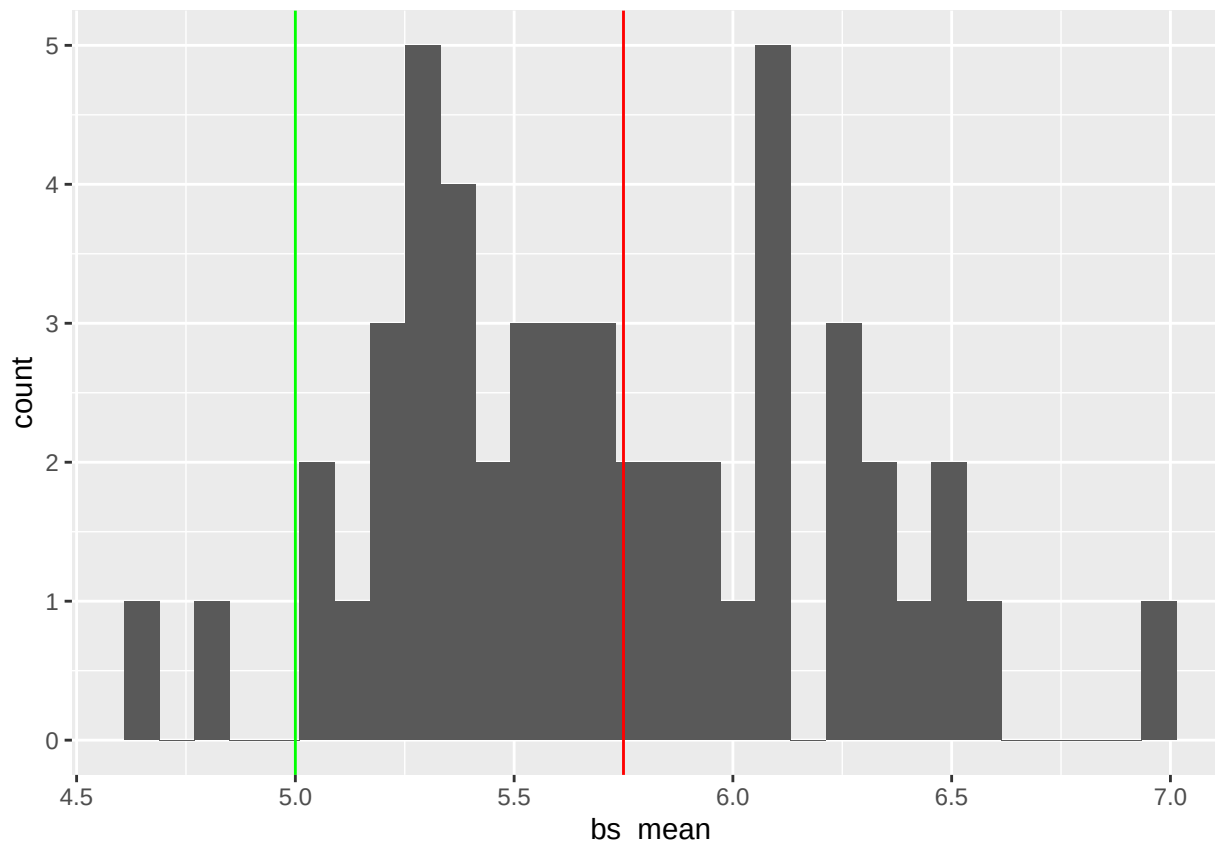
Use your bootstrap function from Section 2 to generate 50 bootstrap means.

```
bootstrap <- function(sample) {
  mean(sample(sample, replace = TRUE))
}
bs_mean <- replicate(50, bootstrap(fake_data1))
#print(bs_mean)
```

Plot your bootstrap means. Add a vertical line in green where your true mean is (the one you used in `rnorm()` to generate the data). Add a vertical line in red where your sample mean is.

```
data.frame(bs_mean) |>
  ggplot(aes(x=bs_mean))+
  geom_histogram()+
  geom_vline(xintercept = 5, color="green")+
  geom_vline(xintercept = mean(fake_data1), color="red")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



Repeat the above steps (the bootstrap replicates and plotting) but this time, with 500 replications. Is your bootstrap distribution's center closer to the actual mean?

Written answer here.

```
bootstrap <- function(sample) {
  mean(sample(sample, replace = TRUE))
}
bs_mean1 <- replicate(500, bootstrap(fake_data1))
print(bs_mean1)
```

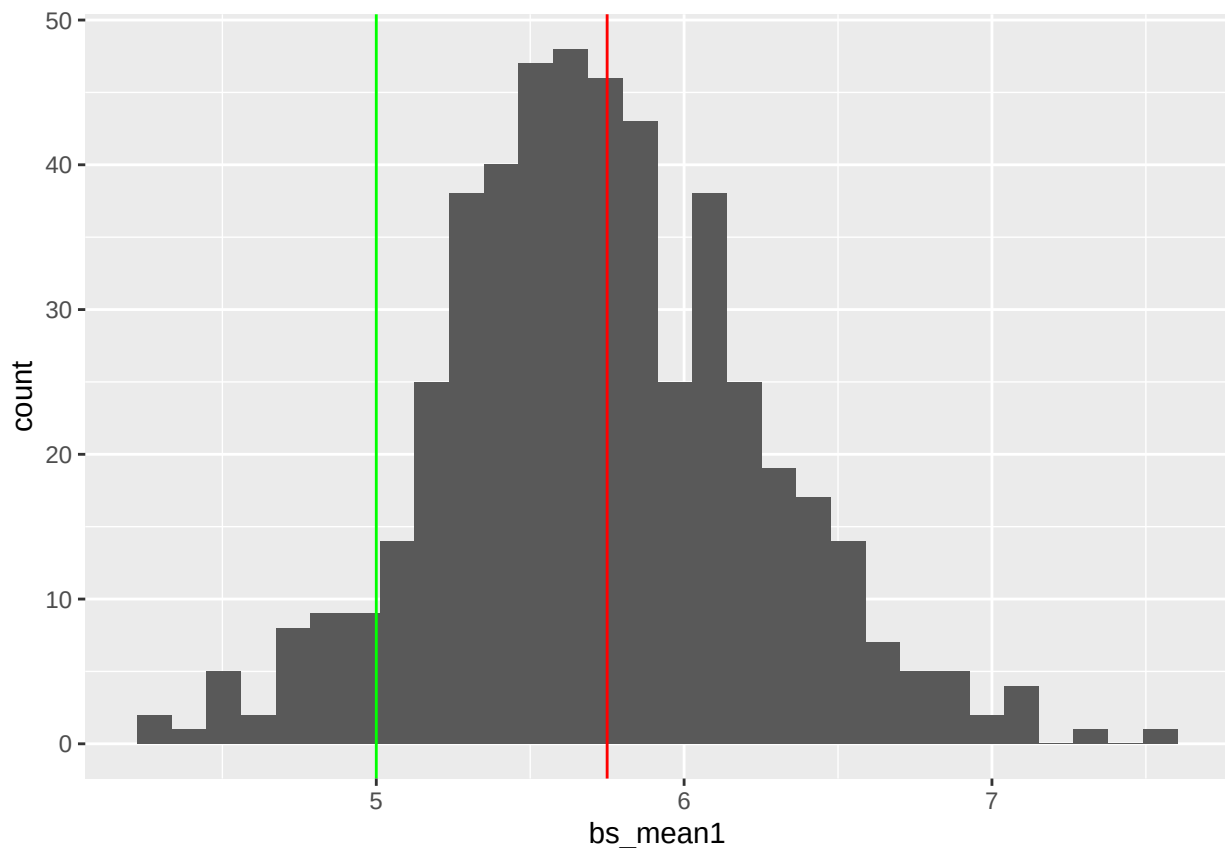
```
## [1] 5.780896 6.307285 6.132019 5.059384 5.567991 6.117771 5.786651 5.319784
## [9] 5.768557 5.487618 6.178187 5.713882 5.843349 7.497497 6.154571 4.766110
## [17] 5.222860 5.711053 5.432593 5.132186 5.403324 6.395644 5.967071 5.828567
## [25] 5.413998 5.773046 5.588218 5.404083 5.904713 5.393417 6.246651 5.843008
## [33] 5.000662 5.865268 5.343498 5.730186 6.064529 5.772874 5.074206 6.480266
## [41] 6.618941 6.223854 5.287843 5.275956 5.386879 5.478643 5.322149 4.861488
## [49] 5.676864 5.967045 6.502553 5.788104 5.815637 5.254553 5.595554 6.289546
## [57] 5.061881 5.129496 5.493470 5.671998 5.130521 6.390271 6.135510 6.015977
## [65] 5.952511 5.338087 7.148590 6.368597 5.715199 6.050135 5.915548 4.527049
## [73] 5.209163 6.327700 5.670081 5.696962 7.336685 6.925429 5.478278 5.489948
## [81] 5.719849 5.162496 5.458237 5.567873 5.533471 5.660402 4.901877 5.577541
## [89] 5.630381 5.304166 5.327766 5.710236 6.245898 6.316401 5.403125 6.546684
## [97] 5.867004 5.640564 5.259582 6.128671 4.652613 5.612605 5.848626 5.350481
## [105] 5.902792 5.970463 5.827146 5.595740 5.184217 5.656091 5.816025 6.634507
## [113] 5.855759 5.380887 6.897018 6.325146 5.394487 5.518997 6.421685 4.959720
```



```
## [121] 5.467880 6.266747 5.560635 5.361593 6.672952 4.231284 5.859510 5.908308
## [129] 5.937213 5.697832 5.929110 5.977787 5.471581 5.482791 5.793146 5.534616
## [137] 7.040072 5.878198 5.541949 6.022958 6.410065 5.912890 5.711193 6.205035
## [145] 5.451697 5.616073 5.511965 6.170452 5.496220 6.520522 6.424495 5.757276
## [153] 5.526010 4.884168 5.758673 5.442777 7.002776 6.458293 5.820289 5.555399
## [161] 5.335733 5.626472 5.401748 5.348511 6.930484 5.626419 5.365073 5.348040
## [169] 5.730383 5.763029 5.412397 4.704189 6.841360 5.780645 4.804722 5.809018
## [177] 5.300464 5.390316 5.816150 5.146304 5.485300 5.233592 5.689903 6.102598
## [185] 5.505973 6.111108 5.804081 4.566453 6.424161 6.052273 6.457067 6.257851
## [193] 5.528562 5.413902 5.173531 6.110466 5.500131 5.318122 6.553407 5.931126
## [201] 5.330634 6.494258 5.685861 6.159117 5.635252 6.039668 6.147519 5.321611
## [209] 6.598287 5.870506 6.332498 5.298542 6.367084 5.856657 5.899937 5.606209
## [217] 5.815451 5.382476 5.140978 5.798012 5.183900 6.195712 5.334709 5.766021
## [225] 6.090760 6.237099 6.189967 5.633915 5.252430 5.598134 5.674018 5.663719
## [233] 5.549199 5.304260 4.535567 5.726848 5.640655 6.618743 5.556213 5.726862
## [241] 6.198016 6.173336 6.035985 5.790580 5.125735 6.128260 5.450307 4.879501
## [249] 6.779772 5.718558 5.672291 5.804961 6.035494 6.582444 6.741779 5.678213
## [257] 5.637276 5.827305 5.803909 5.640896 6.055240 5.327789 4.866826 5.172807
## [265] 6.358374 5.499006 5.262082 5.205958 5.094511 5.312758 6.085542 5.487256
## [273] 6.426648 5.684190 6.081312 5.403675 5.411505 6.130198 5.873149 5.247324
## [281] 6.259980 5.233608 6.495800 5.826102 5.519064 6.324931 5.103538 5.186663
## [289] 4.899116 5.955798 6.374239 5.214248 6.849336 6.626292 5.946681 5.838513
## [297] 4.705187 5.036082 5.724387 4.949839 5.918443 5.504068 6.052595 5.859089
## [305] 5.452278 6.673860 4.967365 6.077433 5.823658 6.116044 5.420216 5.163860
## [313] 5.342235 5.491762 5.269586 5.536741 5.724192 5.677798 5.982799 5.621543
## [321] 5.591801 5.819110 5.461587 6.029074 5.621352 5.582538 5.618383 6.710225
## [329] 7.090762 5.446724 7.147959 5.929266 5.424749 6.151847 5.093011 5.741481
## [337] 6.215084 5.432118 5.363664 5.542733 5.938806 5.357267 5.673333 5.967188
## [345] 5.553329 6.226212 5.395063 5.748528 6.069779 5.245480 5.334431 6.549757
## [353] 4.819151 5.279269 5.655192 6.042162 5.612746 6.359327 5.279182 6.020865
## [361] 5.500766 5.695062 5.787585 6.432340 6.234874 5.688586 6.157971 4.380437
## [369] 4.941187 6.562303 5.302429 4.732186 5.602198 5.513891 5.239749 6.050975
## [377] 5.991787 6.399001 5.559878 5.416694 6.289595 4.785266 4.789273 6.298268
## [385] 5.239963 4.934652 5.518764 5.823596 6.556233 6.165944 5.689389 5.583531
## [393] 5.278841 5.679270 5.403532 5.333139 5.627382 5.947038 6.179777 4.878610
## [401] 5.572871 4.706881 6.715197 6.392453 5.568822 5.151894 5.733939 5.837240
## [409] 5.505442 5.669079 5.750841 5.979066 5.481841 5.344042 5.698425 5.618739
## [417] 6.489052 5.922297 5.536814 5.191591 5.454765 6.206983 6.188044 5.954958
## [425] 5.862875 5.469025 6.290300 5.560424 4.555995 4.772985 6.363115 6.134707
## [433] 5.491878 5.054359 5.737934 5.454340 5.362223 5.803766 6.712197 6.555471
## [441] 5.269600 4.549693 6.141998 5.016132 6.503892 6.083030 5.104084 6.085552
## [449] 6.281493 6.046986 5.715623 6.229970 5.739179 4.752546 5.773671 5.613478
## [457] 5.835013 6.106626 6.394859 6.079344 5.699154 5.907686 6.464801 6.351806
## [465] 5.820213 5.760109 6.136675 5.395893 5.702828 5.027164 6.107702 5.397825
## [473] 6.137254 5.824522 6.074900 5.526366 5.855786 5.955005 5.128378 5.092043
## [481] 5.658949 5.153799 6.819260 4.293705 5.561821 4.493963 6.308830 5.665316
## [489] 4.917283 6.092314 6.048242 5.806607 4.942878 5.075157 5.134771 5.061154
## [497] 5.402931 5.605853 5.577653 5.147537
```

```
data.frame(bs_mean1) |>
  ggplot(aes(x=bs_mean1))+
  geom_histogram()+
  geom_vline(xintercept = 5, color="green")+
  geom_vline(xintercept = mean(fake_data1), color="red")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



Repeat this again with 1000 replications. Did this make a drastic difference in the distribution's center?

Written answer here.

```
bootstrap <- function(sample) {
  mean(sample(sample, replace = TRUE))
}
bs_mean2 <- replicate(1000, bootstrap(fake_data1))
print(bs_mean2)
```

```
##      [1] 6.098256 6.419874 7.119156 5.408195 5.513568 6.361911 6.435507 6.465189
##      [9] 5.832608 5.929003 5.411265 4.959885 5.825657 6.041676 6.599851 5.920064
##     [17] 5.477351 5.658411 6.116884 6.728242 5.585318 6.238525 6.232363 5.715241
##     [25] 6.084903 4.779709 5.500194 5.136098 5.480179 5.243938 6.160578 4.721864
##     [33] 5.919433 6.587351 6.096792 6.510231 5.353790 6.072637 5.195996 5.545699
##     [41] 6.595534 5.410180 5.407818 5.569529 5.878637 5.571267 6.756795 6.052881
##     [49] 5.887778 5.782266 6.280907 5.849147 6.169243 6.004564 5.342580 5.053994
##     [57] 5.460835 5.911989 6.408571 4.857529 5.843059 4.942492 5.745826 6.925736
##     [65] 5.278443 5.913327 6.008857 5.848820 6.580956 6.375962 5.574571 5.429122
##     [73] 5.652412 5.286212 6.062109 5.424397 7.006475 5.213553 6.398750 5.225895
##     [81] 6.406602 5.125306 5.801544 5.868212 5.287882 4.982556 6.306480 5.334018
##     [89] 5.794642 5.835523 5.613981 5.236404 5.895067 5.227484 5.882864 5.614709
##     [97] 5.718284 6.865864 5.767422 6.161066 6.455755 5.277008 4.679762 7.167695
```

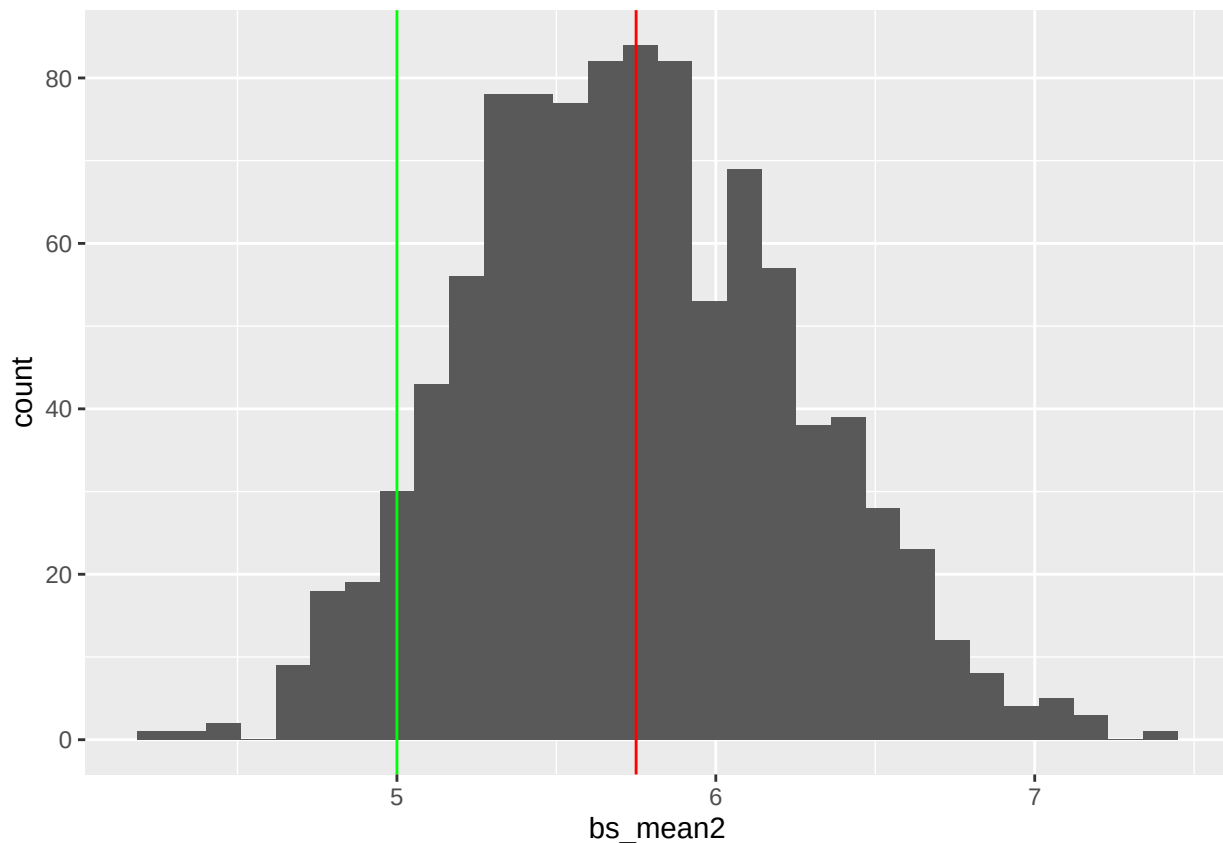
[105] 5.666938 5.632485 6.241865 5.800974 5.746418 5.351267 6.389836 5.652316
 ## [113] 5.247663 5.997501 5.467763 5.727625 6.315890 5.425841 4.991080 5.914340
 ## [121] 5.649322 4.923302 4.950532 5.641483 5.721670 5.405680 5.484213 4.855757
 ## [129] 5.791266 6.211017 6.563637 4.831433 5.489725 4.909554 5.553847 5.092728
 ## [137] 5.725848 5.342819 6.011254 5.130992 5.697635 5.885010 5.296112 5.801999
 ## [145] 5.144091 5.633187 6.569925 6.097071 6.772177 6.021186 5.664541 5.434521
 ## [153] 5.555461 5.452661 5.623279 4.812826 5.681770 5.436864 4.412499 5.515726
 ## [161] 5.482299 6.211849 5.227974 6.342941 6.212467 5.793285 6.215652 6.242231
 ## [169] 6.195973 6.046861 5.348717 6.007207 6.341525 5.583185 6.324843 4.970338
 ## [177] 6.171004 6.227470 5.851231 6.162323 6.065222 5.317698 5.660858 5.907192
 ## [185] 5.793159 5.303328 5.943271 5.339261 5.117206 5.491649 5.134880 5.215259
 ## [193] 5.690573 6.418381 4.867330 5.565840 5.552894 5.568392 6.228422 5.626312
 ## [201] 5.527357 5.602026 5.618163 6.170826 5.548036 6.555138 4.859159 6.112926
 ## [209] 5.396257 5.482279 5.440785 5.934292 4.810908 4.923348 5.272468 6.358380
 ## [217] 6.529483 5.848432 5.492012 6.362880 5.813139 5.502942 5.653994 5.875348
 ## [225] 6.046371 6.003161 5.885746 5.387382 6.974126 7.199324 6.310765 5.060588
 ## [233] 5.442248 5.644644 5.500639 5.906413 7.346383 6.084186 5.860847 5.753589
 ## [241] 5.453378 4.994114 5.633245 5.414788 5.274961 6.079253 5.510121 4.348276
 ## [249] 5.286579 5.127642 5.612465 5.882460 5.097426 5.636438 6.408873 6.318934
 ## [257] 5.987400 5.566662 5.985368 5.732898 5.529804 5.542495 5.418021 6.109290
 ## [265] 5.841197 5.161207 5.463018 5.414140 5.742424 5.602940 6.259426 5.528612
 ## [273] 4.690548 5.421308 5.737126 5.881768 6.142994 5.840591 5.688624 6.388279
 ## [281] 6.392604 5.588996 6.244369 4.498777 4.818885 5.698586 5.083831 5.757947
 ## [289] 5.900737 4.804719 5.997984 6.545455 5.111830 5.561878 5.125452 5.768290
 ## [297] 5.982088 6.072748 6.184342 5.652722 5.810267 6.353312 5.809844 6.097495
 ## [305] 4.645323 5.374331 5.297749 5.237381 6.137651 5.444206 6.231967 5.605585
 ## [313] 6.085144 6.523816 6.363134 5.365459 5.129340 6.104167 5.878240 5.904328
 ## [321] 5.657385 5.303491 6.005669 6.269044 4.927986 5.229767 5.407998 5.836902
 ## [329] 6.320718 5.368318 5.760915 6.358408 6.139495 5.487957 5.496561 5.266185
 ## [337] 5.620872 5.433878 6.001639 6.521736 6.167066 5.284673 5.925922 5.453244
 ## [345] 5.342654 6.367000 5.971208 6.676429 5.836246 5.932208 6.444048 6.389185
 ## [353] 5.233883 5.786732 5.070191 5.096159 5.675710 4.820331 5.574365 5.049756
 ## [361] 6.501957 5.775482 5.790647 5.965608 5.611460 5.759045 5.132525 5.423149
 ## [369] 5.277457 5.908541 5.687626 5.202713 4.821730 5.315905 5.099918 5.838403
 ## [377] 6.048269 6.016398 6.214509 5.862736 5.747633 5.685425 5.504408 4.787118
 ## [385] 5.431058 4.718847 6.295153 4.708390 5.290720 5.363001 5.820607 6.377299
 ## [393] 5.571592 6.016547 5.037605 5.561345 6.761175 6.388500 6.430671 5.694525
 ## [401] 6.460132 5.550359 6.078654 5.353832 5.820577 5.751456 6.665629 5.715901
 ## [409] 6.476196 5.416725 6.616457 6.444686 5.657010 5.189918 5.329675 5.868715
 ## [417] 5.879851 4.193479 6.059507 6.136790 4.923091 5.871347 6.030363 5.602765
 ## [425] 6.170569 5.733710 5.191773 6.532983 5.508872 5.060111 6.135036 6.518412
 ## [433] 5.305411 5.011070 5.821299 6.800282 6.043123 6.222904 6.829252 6.724211
 ## [441] 6.680273 5.700264 5.307419 6.237042 6.782103 5.155886 6.247374 5.790850
 ## [449] 5.067144 5.049385 6.392273 5.914630 5.696709 4.954893 5.483538 6.214998
 ## [457] 6.107110 6.583136 5.142151 5.362212 5.762115 5.039629 5.166301 5.279486
 ## [465] 5.900363 5.880074 6.411474 6.074512 5.728944 4.708252 5.891832 6.442820
 ## [473] 5.331995 5.745696 6.187191 5.615946 5.485930 5.642624 5.223962 4.897915
 ## [481] 5.239520 5.054451 5.908062 5.691658 5.629694 6.111128 5.414025 5.691997
 ## [489] 6.209808 5.726386 5.637433 5.303300 5.130523 5.538173 5.497421 5.445995
 ## [497] 5.326562 5.478733 5.412235 5.956421 5.093039 5.795089 6.897578 4.791114
 ## [505] 5.380607 5.327124 6.030913 5.893232 5.759377 6.084137 6.249562 5.314472
 ## [513] 6.219097 5.314465 6.273969 5.487431 5.945378 4.965452 5.393812 5.562747
 ## [521] 5.401358 6.294157 4.797819 5.265563 5.483368 6.028893 5.141324 6.034418
 ## [529] 5.768927 5.990995 5.679905 5.048763 5.504022 6.885332 5.648719 6.120525

[537] 5.243489 5.705139 5.276900 5.761359 5.264656 6.097945 6.381645 6.136148
 ## [545] 6.227160 5.669319 6.393091 5.047149 5.370364 5.774080 5.848159 6.547320
 ## [553] 5.156112 6.168351 5.622993 4.809716 5.029585 6.850146 4.971113 6.537349
 ## [561] 5.957156 5.365839 5.383254 5.162834 5.343073 5.451897 5.322252 5.851132
 ## [569] 5.287059 6.303738 6.535785 5.577621 5.443789 6.314147 6.167041 5.982684
 ## [577] 5.776152 5.479405 6.837386 5.758794 5.523435 6.119004 5.921165 5.406904
 ## [585] 6.337081 6.244033 4.783691 6.133501 5.417500 5.854948 5.515746 5.670632
 ## [593] 5.278015 5.224283 6.104101 5.999403 5.964995 4.788187 5.684276 5.759815
 ## [601] 4.845413 6.012314 6.365731 6.460501 5.352394 5.703513 4.724439 6.035738
 ## [609] 6.282599 5.195742 5.716279 5.468387 6.675891 5.336411 6.234937 5.558598
 ## [617] 5.401901 6.269784 5.279504 6.036677 5.205951 6.130274 5.617170 5.822096
 ## [625] 4.969160 5.844178 6.153239 5.858476 6.146950 6.683646 6.204484 5.681050
 ## [633] 5.336902 6.065447 5.264042 5.768162 5.774995 7.061469 5.568272 6.425181
 ## [641] 7.078367 5.530040 5.796494 5.877226 6.774407 5.168531 5.871239 6.035559
 ## [649] 6.842902 5.989101 5.372595 6.207324 4.933804 6.102579 6.378399 4.914788
 ## [657] 5.623663 5.181585 5.311505 6.009171 6.224630 5.689469 5.865544 5.728741
 ## [665] 5.652545 4.742495 5.452411 4.960090 4.914926 6.135321 7.048572 6.041590
 ## [673] 4.990188 5.524046 6.043557 5.159180 5.288888 5.819513 5.535899 4.933182
 ## [681] 5.646809 6.036232 5.716820 5.781053 4.804630 5.713679 5.857582 5.209375
 ## [689] 5.476334 5.972146 5.574808 5.470897 6.078584 5.888671 6.056445 5.234432
 ## [697] 5.355272 5.545546 6.212530 5.778382 5.796160 6.191547 5.711083 6.046164
 ## [705] 5.721201 6.479767 5.975227 4.629105 5.881041 5.256135 5.232762 5.772339
 ## [713] 5.952835 6.027556 5.770560 5.810726 6.173763 5.324348 6.295198 5.587971
 ## [721] 5.438964 5.471145 6.155859 5.946779 5.429665 6.330780 5.926359 5.491684
 ## [729] 5.208731 5.689987 6.759036 5.279001 6.342990 5.703959 5.866759 5.236889
 ## [737] 5.756513 5.856334 5.314230 5.595075 5.258486 4.867462 6.620203 5.490447
 ## [745] 5.858774 5.638618 5.574965 5.813867 5.771157 5.063795 5.568920 5.649112
 ## [753] 5.607597 5.829665 5.943542 5.714223 5.461159 5.493049 5.506034 4.880900
 ## [761] 5.339783 6.007140 6.503083 5.710995 5.223676 5.282416 5.034401 6.334802
 ## [769] 6.326610 6.630668 4.976446 5.470312 5.463135 6.540049 5.225465 6.092798
 ## [777] 6.626057 5.573278 5.396843 6.175890 6.182010 5.651176 6.071016 5.398749
 ## [785] 6.351416 5.287960 6.324346 5.312464 5.399609 6.267430 5.607423 5.015327
 ## [793] 6.124002 6.090502 5.016136 6.140183 5.781931 5.866381 6.609799 5.717779
 ## [801] 5.113045 6.644315 6.035131 5.877291 5.890780 6.589133 5.764379 6.597001
 ## [809] 5.762395 6.737497 6.266533 5.635941 6.525721 6.406784 5.657406 5.217410
 ## [817] 6.164498 5.904616 6.177717 6.505463 5.196129 5.194737 5.317335 5.109038
 ## [825] 5.284644 5.684535 5.761631 6.099809 6.358160 5.584572 6.561509 5.371611
 ## [833] 5.719587 5.978378 5.497228 5.913610 6.483676 6.525042 5.763231 5.676391
 ## [841] 5.650961 5.472580 5.249486 5.580482 5.764154 6.098626 5.109087 5.416017
 ## [849] 5.600022 5.480817 5.297971 5.759050 6.625730 5.532647 5.293590 5.165810
 ## [857] 6.470079 5.968820 6.407342 5.729823 6.591876 5.180080 5.544773 5.858468
 ## [865] 4.964410 5.764265 5.685596 5.409633 5.658118 5.207649 6.062695 6.785611
 ## [873] 5.749369 5.453681 5.092593 5.526776 5.237996 5.115970 5.393574 5.999580
 ## [881] 5.873840 6.326283 6.386181 5.677496 5.966948 5.187908 5.181805 5.688306
 ## [889] 5.851817 5.374254 7.191429 5.725649 5.212263 5.329610 5.570186 5.140983
 ## [897] 6.601993 5.887241 5.169097 6.041400 5.218507 6.336415 5.477039 5.563421
 ## [905] 5.173363 5.774655 6.565956 4.987053 6.554379 5.154056 5.676859 5.965888
 ## [913] 6.253001 6.461419 7.021444 5.598457 6.912746 6.061968 5.530803 5.585055
 ## [921] 6.060285 6.114286 5.503736 6.380759 6.305688 6.005686 6.135515 5.556759
 ## [929] 6.316006 6.222578 5.321028 5.994864 6.084445 5.041411 6.736396 5.661566
 ## [937] 5.423119 5.278608 5.298922 6.189477 5.486224 6.687852 6.165750 5.360534
 ## [945] 5.895700 5.082111 5.447001 6.007509 5.187460 5.351617 6.209827 5.121254
 ## [953] 5.267894 5.681016 5.849550 5.504837 5.242308 5.897063 5.642450 5.057627
 ## [961] 4.729828 5.962665 6.465000 5.697064 5.803295 5.375791 5.833256 5.519537

```
## [969] 5.877445 6.176054 5.533560 6.607335 5.157267 5.552188 5.956422 6.550216
## [977] 5.818969 5.072810 6.334291 5.053399 5.059122 5.579161 5.917626 6.155134
## [985] 5.351075 5.564832 5.334317 6.470253 6.098610 5.406597 4.869732 4.745680
## [993] 5.290082 6.114854 5.808883 5.585461 5.236409 6.640989 5.715835 6.187816
```

```
data.frame(bs_mean2) |>
  ggplot(aes(x=bs_mean2))+
  geom_histogram()+
  geom_vline(xintercept = 5, color="green")+
  geom_vline(xintercept = mean(fake_data1), color="red")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



Task 2: Create a bootstrap simulation using 100 datapoints from a normal distribution. Create a single bootstrap distribution using 500 replicates and plot like above.

```
fake_data1 <- rnorm(100, mean = 5, sd = 3)
bootstrap <- function(sample) {
  mean(sample(sample, replace = TRUE))
}
bs_mean3 <- replicate(500, bootstrap(fake_data1))
print(bs_mean3)
```

```
## [1] 4.983862 4.859252 5.104370 4.513284 4.665707 5.228442 4.575528 4.745073
```

```

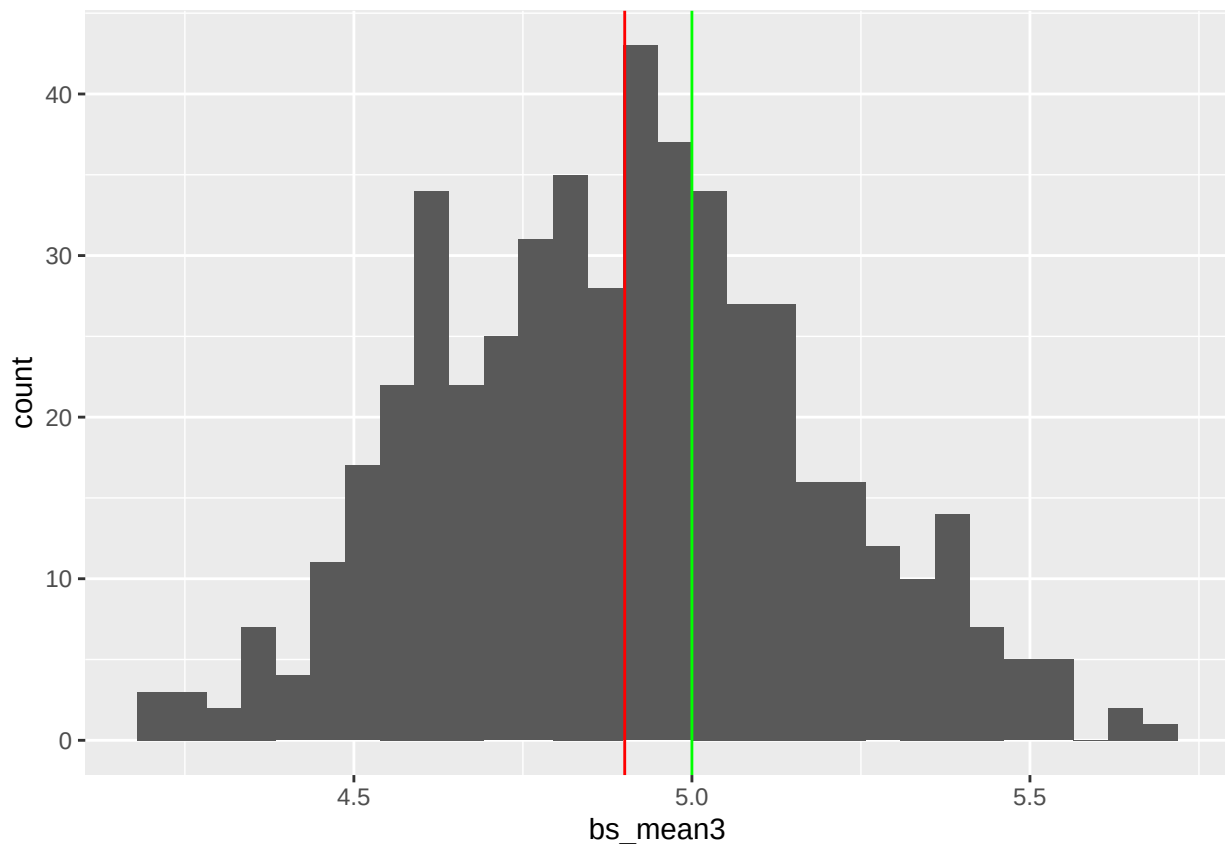
## [9] 4.855572 4.828609 5.206988 4.379006 4.715509 5.069014 4.841859 4.788756
## [17] 5.113053 4.862677 5.394602 4.944054 5.165358 5.325292 5.038216 4.918402
## [25] 4.426621 4.826563 4.538112 4.894045 4.887115 4.841371 5.034235 5.073336
## [33] 4.600824 4.642619 5.126531 5.179228 4.717949 5.241176 5.269242 4.668591
## [41] 5.000477 4.904324 4.822128 4.559146 5.263770 4.855695 5.333377 4.974674
## [49] 4.641394 5.171151 5.675640 5.003674 5.450694 4.363190 5.092050 5.028376
## [57] 4.620793 5.040383 4.622071 4.592930 4.619855 5.110662 5.025859 4.992062
## [65] 5.407464 5.136198 5.025697 4.847894 4.963251 4.943986 4.760131 4.566037
## [73] 5.190276 4.813384 4.815145 4.934890 4.843435 5.313122 4.777635 5.264346
## [81] 4.706267 5.373099 4.952948 4.522757 4.905815 4.753603 4.731348 5.224604
## [89] 4.505600 4.784775 5.227450 4.935604 5.016679 4.675389 4.658867 5.233748
## [97] 4.883568 4.795301 5.534453 5.024787 4.902155 4.894902 5.044721 4.813956
## [105] 4.599675 4.516507 5.062753 4.956249 4.761851 5.095882 4.673324 5.234273
## [113] 4.938678 4.785735 5.104620 4.618974 4.953846 4.497694 4.583179 5.394534
## [121] 4.944680 4.728335 5.465708 5.513231 5.422883 4.568084 5.454774 5.005670
## [129] 4.922232 5.112565 4.916692 4.850174 4.769817 5.036376 5.174447 4.867819
## [137] 5.300009 4.437335 4.689914 4.922125 4.464536 4.548991 4.949999 5.119229
## [145] 5.146796 4.331509 4.269403 4.628626 4.991515 4.957172 5.421587 4.916524
## [153] 4.915662 5.055489 4.751195 4.515134 4.946702 5.390514 4.840209 5.079236
## [161] 5.102088 5.023758 5.263580 4.945049 4.557179 5.649442 4.503684 5.018089
## [169] 4.870112 4.822671 4.589476 4.543638 4.982016 4.856581 4.763511 4.745888
## [177] 4.523243 5.051373 4.822648 4.593850 4.850070 4.444922 4.666606 4.921453
## [185] 4.957267 4.521439 5.195066 4.607800 4.958767 5.033032 5.321191 4.940448
## [193] 5.061376 4.809389 4.998986 4.700379 4.834281 4.558017 4.913188 4.817035
## [201] 4.922919 5.145122 4.218616 5.363356 4.609316 4.610260 5.104271 4.887199
## [209] 4.855519 4.725560 4.661380 4.624274 5.009063 4.671566 5.116537 4.627222
## [217] 5.084971 4.493975 4.804098 4.802910 4.852855 4.823450 4.988686 5.231463
## [225] 4.814781 4.946761 5.238471 4.772761 4.947621 5.068506 5.311497 4.908232
## [233] 5.290231 4.976647 4.435959 5.042676 4.476452 5.048598 5.416737 4.800806
## [241] 5.244782 4.986039 5.532554 5.234219 4.787034 5.219914 4.563453 4.805803
## [249] 4.764738 4.995653 4.550565 5.070859 4.684912 5.185469 4.881640 4.632701
## [257] 4.473151 5.318937 5.190470 4.991298 5.295503 5.149917 4.453971 5.064094
## [265] 5.031169 4.972199 4.939019 5.352702 5.088701 4.784961 4.373068 4.977729
## [273] 4.287853 4.957820 5.141854 5.220390 4.748568 4.625036 4.816392 5.106517
## [281] 4.546288 4.843111 4.628999 4.934996 5.123025 5.478603 4.920784 4.953398
## [289] 4.805772 5.409983 4.593842 5.293945 5.408083 4.769584 5.186269 5.254081
## [297] 4.957613 5.481645 5.023784 4.821278 5.119540 4.920912 4.629216 4.990954
## [305] 5.019099 5.399058 4.972962 5.019860 5.137649 4.637720 4.845352 4.735262
## [313] 4.618603 5.053974 5.285403 4.911476 4.404489 4.953005 4.525645 5.000770
## [321] 4.238657 4.960627 4.862065 5.134659 5.118036 4.715523 4.875091 4.948851
## [329] 5.262049 4.591760 5.006904 4.942614 4.781265 5.357525 5.005218 4.693042
## [337] 4.736947 5.160490 4.727683 4.657315 4.205295 4.672177 5.368723 5.251836
## [345] 5.177514 4.949678 4.791535 5.322452 4.916853 5.385200 4.863988 4.633224
## [353] 4.536155 4.751723 5.041765 4.758525 4.564907 4.372657 5.054970 5.006506
## [361] 5.294132 4.725423 4.401438 4.438031 4.642931 4.986971 5.464709 4.854299
## [369] 4.684995 5.543351 5.099600 5.222638 5.042696 5.334026 5.551018 4.928261
## [377] 5.420698 5.192858 5.068870 5.144324 5.118711 4.937792 4.887826 4.562619
## [385] 5.268510 4.785107 4.915917 4.881943 4.784271 5.175471 4.541048 4.699607
## [393] 4.380211 4.805643 4.670180 5.033619 4.499574 4.930372 4.509915 5.199612
## [401] 4.606156 4.731150 4.810081 4.634278 4.733569 4.935408 5.085483 4.846301
## [409] 5.467381 4.773899 4.898023 4.461018 4.551374 5.158543 4.616503 4.928037
## [417] 4.700880 4.771591 4.725844 5.095838 5.056519 4.968088 4.955888 4.624224
## [425] 4.786976 4.950556 4.937741 4.697826 4.883512 4.559677 4.826647 4.581094
## [433] 4.765869 5.130851 5.158901 4.738027 4.623107 4.808903 4.545467 4.840241

```

```
## [441] 5.083084 4.560767 5.095008 4.685938 4.905459 5.113827 4.975571 5.010155
## [449] 4.926999 4.622802 5.051596 4.736398 5.110124 4.697368 4.806412 4.793859
## [457] 5.053338 4.595244 5.418643 4.921425 4.615593 4.337422 5.374524 5.395431
## [465] 4.443353 4.383287 4.992079 4.459158 4.678411 5.116296 4.873631 4.661253
## [473] 5.654891 4.880824 4.276263 4.516589 4.777915 4.675442 4.804095 4.639795
## [481] 4.467069 4.742738 4.907848 4.591409 4.647481 5.062992 5.094212 4.837820
## [489] 4.188421 4.611276 5.373479 5.023290 4.994575 4.694407 5.121485 4.766791
## [497] 4.580538 4.495384 4.740679 5.022134
```

```
data.frame(bs_mean3) |>
  ggplot(aes(x=bs_mean3))+
  geom_histogram()+
  geom_vline(xintercept = 5, color="green")+
  geom_vline(xintercept = mean(fake_data1), color="red")
```

'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.



Did the increased sample size help with the accuracy of the bootstrap distribution?

Written answer here.

Section 6: Confidence Intervals

Generate another fake dataset of 100 datapoints from a normal distribution with mean 70 and sd 4.

```
fake_data2 <- rnorm(100, mean=70, sd=4)
```

Generate a bootstrap distribution using 500 replicates.

```
bootstrap <- function(sample) {  
  mean(sample(sample, replace = TRUE))  
}  
bs_mean4 <- replicate(500, bootstrap(fake_data2))
```

Use `quantile()` to get a 95% confidence interval.

```
lowerbound <- quantile(bs_mean4, 0.025)  
upperbound <- quantile(bs_mean4, 0.975)
```

Plot this distribution. Use two vertical lines to represent the bounds of your above interval.

```
df <- data.frame(bs_mean4 = bs_mean4)  
  
ggplot(df, aes(x = bs_mean4)) +  
  geom_histogram(fill = "lightblue") +  
  geom_vline(xintercept = lowerbound, color = "red") +  
  geom_vline(xintercept = upperbound, color = "blue")
```

'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.



Instead, let's assume CLT holds. Use the sample mean and sample standard deviation to create your interval. How does this compare to your bootstrap interval?

We are increasing sample size rather than replacing sample.

```
sample_mean <- mean(bs_mean2)
sample_sd <- sd(bs_mean2)
n <- length(bs_mean2)

alpha <- 0.05
z <- qnorm(1 - alpha/2)

lowerbound <- sample_mean - z * (sample_sd / sqrt(n))
upperbound <- sample_mean + z * (sample_sd / sqrt(n))
```

Let's say this dataset is the heights of American men. How would you interpret the confidence interval (you may use your bootstrap interval for this question)?

We are confident that between the lower bound and upperbound the overall population of american men's heights are between that interval.

Section 7: Running a t-Test

Generate another fake dataset with 50 points attributed to group A (normal distribution with mean 15 and sd 2) and 50 points attributed to group B (normal distribution with mean 10 and sd 2).

```
group_A <- rnorm(50,15,2)
group_B <- rnorm(50,10,2)
data.frame(values = c(group_A, group_B),
            group = rep(c("A", "B"), each = 50))
```

```
##      values group
## 1  13.775238     A
## 2  13.633903     A
## 3  12.763441     A
## 4  16.455304     A
## 5  11.304373     A
## 6  19.475295     A
## 7  14.213628     A
## 8  12.870299     A
## 9  14.575745     A
## 10 11.426987     A
## 11 16.042342     A
## 12 17.949795     A
## 13 12.826476     A
## 14 18.386256     A
## 15 14.263952     A
## 16 15.288698     A
## 17 12.614916     A
## 18 11.609545     A
## 19 14.351755     A
```

## 20	12.069314	A
## 21	13.361373	A
## 22	17.593037	A
## 23	12.447858	A
## 24	14.329448	A
## 25	15.625106	A
## 26	14.398000	A
## 27	12.807467	A
## 28	15.537893	A
## 29	12.907842	A
## 30	18.452169	A
## 31	13.093345	A
## 32	14.490821	A
## 33	15.786000	A
## 34	16.538193	A
## 35	12.315291	A
## 36	18.210273	A
## 37	14.031499	A
## 38	16.286284	A
## 39	13.840086	A
## 40	14.679995	A
## 41	13.817789	A
## 42	16.446385	A
## 43	15.848657	A
## 44	14.095453	A
## 45	15.051738	A
## 46	17.121458	A
## 47	17.961897	A
## 48	14.134285	A
## 49	12.504956	A
## 50	16.676081	A
## 51	8.360161	B
## 52	11.601238	B
## 53	13.588848	B
## 54	8.571903	B
## 55	11.148360	B
## 56	10.763759	B
## 57	10.338736	B
## 58	9.372650	B
## 59	10.092063	B
## 60	10.645357	B
## 61	8.929609	B
## 62	10.602441	B
## 63	11.046077	B
## 64	10.094208	B
## 65	8.888050	B
## 66	10.261095	B
## 67	10.673603	B
## 68	7.443244	B
## 69	7.152392	B
## 70	7.815759	B
## 71	6.618847	B
## 72	12.240138	B
## 73	8.986739	B

```
## 74 8.288526 B
## 75 10.292065 B
## 76 9.332982 B
## 77 9.202877 B
## 78 8.101920 B
## 79 10.261357 B
## 80 11.197861 B
## 81 10.923259 B
## 82 12.923420 B
## 83 9.089176 B
## 84 6.318283 B
## 85 9.316383 B
## 86 11.123156 B
## 87 11.410398 B
## 88 9.912806 B
## 89 13.467336 B
## 90 9.184945 B
## 91 11.392502 B
## 92 9.076223 B
## 93 8.232880 B
## 94 10.301179 B
## 95 8.080498 B
## 96 11.980205 B
## 97 10.852955 B
## 98 11.602729 B
## 99 15.572418 B
## 100 11.278899 B
```

Find the observed t-statistic using `t.test()`.

Written answer here.

```
group_A <- rnorm(50,15,2)
group_B <- rnorm(50,10,2)
fake_data3 <- data.frame(values = c(group_A, group_B),
                          group = rep(c("A", "B"), each = 50))
t.test(values~group, data=fake_data3)$statistic
```

```
##          t
## 11.92891
```

Use your t-statistic function from Section 2. Do you get the same answer?

The values are nearly identical, but not completely. I believe the reason is because the formula assumes that the variances are equal or accounts for degrees of freedom. Also, there could be a round off error within the variables which added up to a greater error.

```
t_stats = function(x,y) {
  avg1 = mean(x)
  avg2 = mean(y)
  sd1 = sd(x)
  sd2 = sd(y)
  length1 = length(x)
  length2 = length(y)
```

```

t_stats = (avg1-avg2)/(sqrt((sd1)^2/(length1)) + ((sd2)^2/(length2)))
}

print(t_stats(group_A, group_B))

```

```
## [1] 14.04755
```

Modify your bootstrap function to generate bootstrap t-statistics.

```

bootstrap_t <- function(sample) {
  sample$group <- sample(sample$group)
  t.test(values ~ group, data = sample)$statistic
}
bootstrap_t(fake_data3)

```

```
##      t
## 1.293
```

Generate your bootstrap distribution with 500 replicates. Plot this with a vertical line at your observed test statistic.

```

group_A1 <- rnorm(50, 12, 3)
group_B1 <- rnorm(50, 16, 7)

fake_data4 <- data.frame(
  values1 = c(group_A1, group_B1),
  group1 = rep(c("A", "B"), each = 50)
)

t_value <- t.test(values1 ~ group1, data = fake_data4)$statistic

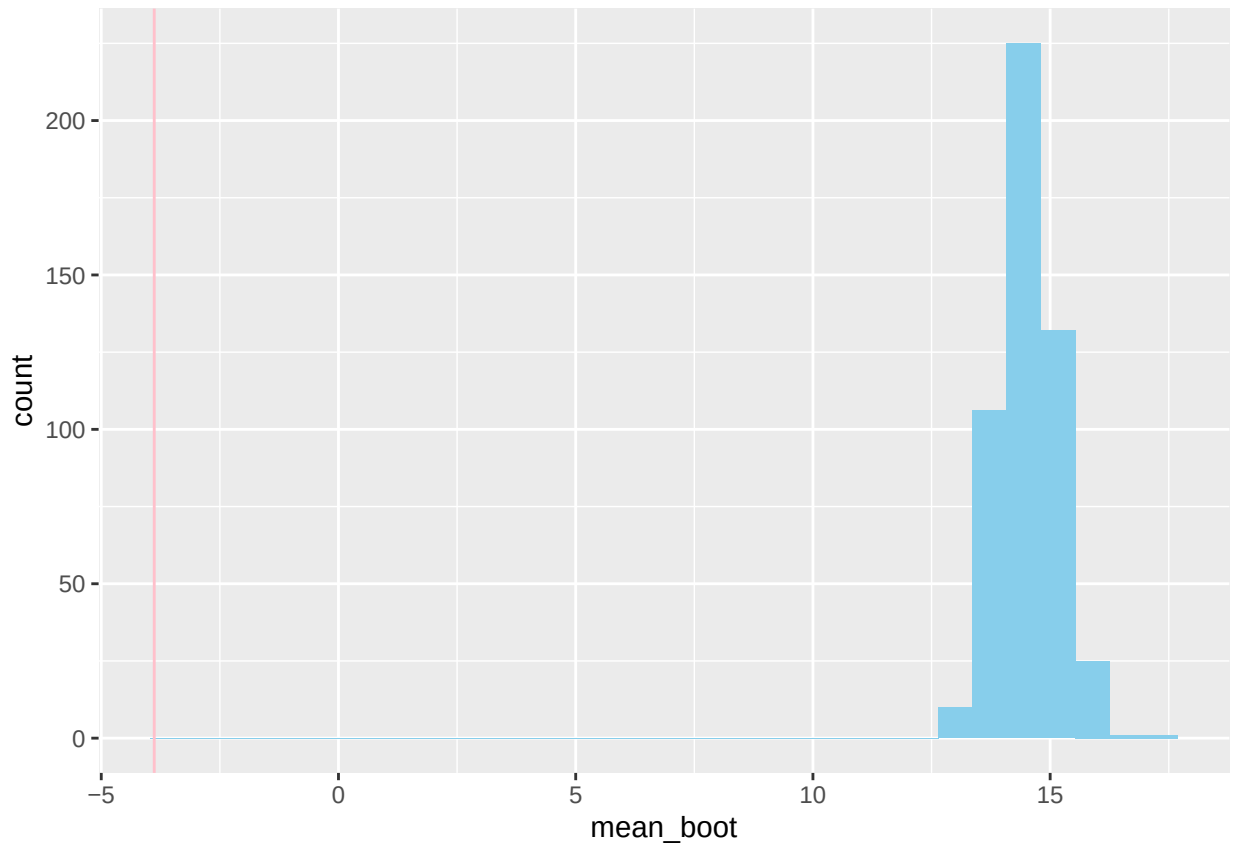
bootstrap <- function(data) {
  sampled <- data[sample(nrow(data), replace = TRUE), ]
  mean(sampled$values1)
}

bs_mean3 <- replicate(500, bootstrap(fake_data4))
df <- data.frame(mean_boot = bs_mean3) # Create df for plotting

ggplot(df, aes(x = mean_boot)) +
  geom_histogram(fill = "skyblue") +
  geom_vline(xintercept = t_value, color = "pink")

```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



Using just your visual reasoning, does this look statistically significant?

It is statistically significant because it shows how the difference between the two groups are not due to chance.

Assume we are conducting a one-way hypothesis test. Write the null hypothesis below.

H_0 : There is no difference within the population mean.

H_A : At least one group mean is different from the others within the population.

Assume $\alpha = 0.05$. Use `quantile()` to determine the rejection threshold.

```
• rejection_thresholds <- quantile(bs_mean3, probs = c(0.025, 0.975))
```

Use R to check if your observed statistic is statistically significant.

```
t_test_result <- t.test(values1 ~ group1, data = fake_data4)
t_test_result$p.value
```

```
## [1] 0.0002591994
```

#Since this is less than our predefined alpha value of 0.05 we conclude that this is statistically significant

Interpret the results of your hypothesis test.

p-value is the probability of an extreme happening and alpha value is the maximum threshold that there is a type 1 error since the p value is less than the alpha value we can assume that the data is not resulting by chance but is actually statistically significant proving our hypothesis test true.

What changes would you make if you were conducting a two-way test?

I would have two bounds so in addition to 0.05 there would also be 0.95.

Section 8: Fitting a Simple Linear Regression

Load the library Lahman.

```
library("Lahman")
```

```
## Warning: package 'Lahman' was built under R version 4.5.1
```

Load the Teams dataset.

```
data("Teams")
```

Filter Teams to only include data from 2000-Present and save back into Teams.

```
Teams <- Teams |>  
  filter(yearID >= 2000)
```

Use the `lm()` function to create a linear regression using earned run average to bring wins.

```
model <- lm(RA ~ W, data=Teams)
```

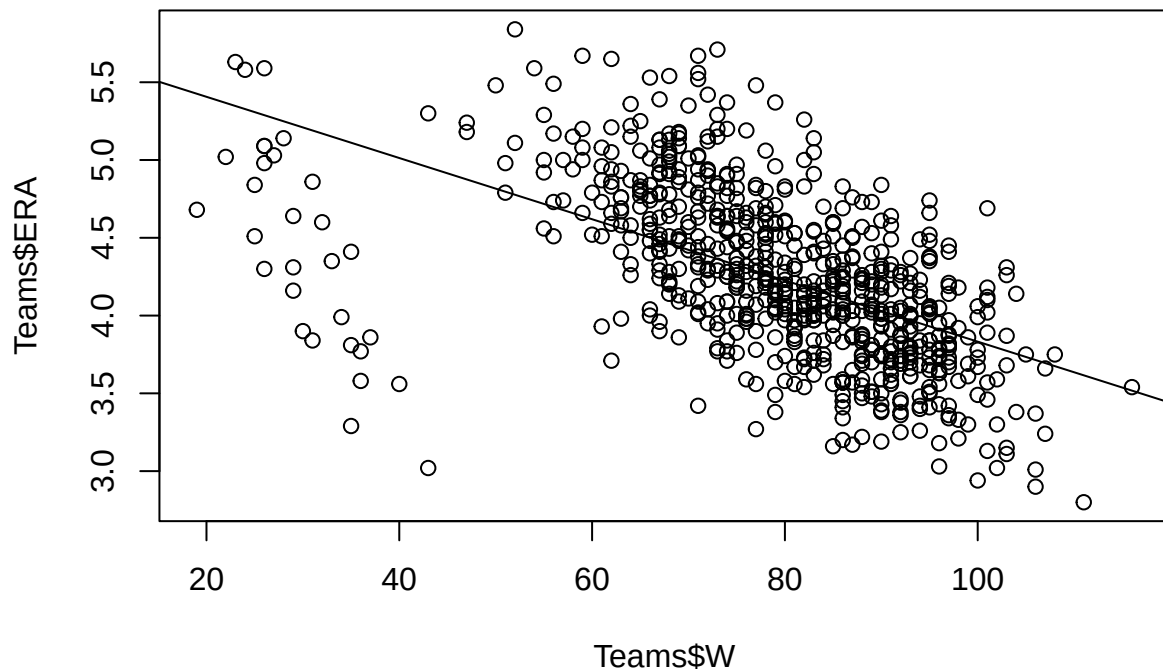
Use `summary()` on the above linear model to get the adjusted R-squared.

```
summary(model)$r.squared
```

```
## [1] 0.01321805
```

Plot earned run average vs wins. Add the line you generated above.

```
model_1 <- lm(ERA ~ W, data=Teams)  
plot(Teams$W, Teams$ERA)  
abline(model_1)
```



How well does your line fit? Include a description of the plot and the R-squared value.

#I would say the line fits pretty well because the line goes through between the cluster of points right
`summary(model_1)$r.squared`

```
## [1] 0.3145505
```

Find the R-squared value of the linear model below. Why is it such a perfect fit? Is this useful?

It is such a perfect fit because the R^2 value is almost 1 this is useful to show that the independent variable is directly explained by the dependent variable.

```
m2 <- lm(W ~ (L+G), data = Teams)
summary(m2)$r.squared
```

```
## [1] 0.9999334
```

Use three variables in a new linear model to get a higher R-squared. Report your adjusted R-squared.

```
m3 <- lm(Rank ~ W + L, data = Teams)
summary(m3)$r.squared
```

```
## [1] 0.762688
```